Current Status of Software Log Analysis at ALMA Observatory

Juan Pablo Gil[a], Nicolás Miranda[a], Mario Garcés[a], and Jorge Avarias[a]

[a]ALMA Observatory, Alonso de Córdova 3107, Vitacura, Santiago, Chile

ABSTRACT

Since the start of operations in 2013, the ALMA Observatory software has become very large in terms of size and complexity requirements for stability, performance and quality have increased. In this work we describe the current status of infrastructure, tools and practices for software log analysis developed over the years to extract insights on the behavior of the system and to identify common pitfalls and points of failure. Thanks to the design and implementation of a logging infrastructure, temporal domain analysis and visualization tools, machine learning techniques and other ad-hoc solutions we have been able to speed up troubleshooting, anticipate issues and have a better understanding of the overall system behavior.

Keywords: Log analysis, Process Mining, DevOps, Data Science, Machine Learning

1. INTRODUCTION

ALMA observatory has been in operation since 2013 but it is still evolving and new capabilities are being introduced every year. At the same time, performance and quality operational requirements are becoming higher, such as the need to deliver increasing amounts of scientific data in fewer time, what leads to the development of better strategies to keep technical downtimes as low as possible by solving emerging problems quickly and anticipating new ones before they have a real impact in observations.

ALMA is organized in operational groups, engineers, software, operators, science specialists that do data collection and reduction in pipelines. The overall observatory process chain from project specifications, data collection to scientific results delivery is controlled by a software suite made by a collaborative efforts among ALMA partners. This software has grew over the years and it is expected to handle an increasing number of boundary cases that emerge from the new features and interaction of a large number of hardware types and subsystems.

The ALMA Software ecosystem groups several kinds of subsystems, developed across three continents, which control the real time operation of the observatory (controlled by what has been called *ONLINE software*) and the processing of data and posterior analysis of the performance of the observation and science analysis process (controlled by *OFFLINE software*). The most reliable source to understand the behaviour of such complex system is through software events which are duly recorded in their respective system logs.

In this document we describe, in the form of a survey, the different ways in which we have addressed some of the challenges of troubleshooting and error detection in the context of the ALMA Software system through software log handling, querying and analysis. We hope this information will be useful to other scientific organizations that perform operation of complex districuted systems and look for better ways to understand their behavior and identify potential pitfalls.

Further author information: (Send correspondence to Juan Pablo Gil)
Juan Pablo Gil: E-mail: juan.gil@alma.cl, Telephone: +56 (2) 2467 6100

2. LOG GENERATION IN THE ALMA SOFTWARE INFRASTRUCTURE

2.1 ACS logging system

The ACS logging subsystem (here should go a reference to logging service document) leverages three systems that have been either already built, or at least well designed and specified. These are:

- CORBA Telecom Log Service for centralizing all log entries generated throughout the system.
- CORBA Notification Service for distributing log entries to interested clients (logs consumers) when the entries are submitted to the centralized logger.
- The mechanism for generating, formatting, filtering and caching log entries by clients, components and containers.
  - C++ logs suppliers use the ACE Logging framework with its C++ API;
  - Java logs suppliers use the standard Java Logging API;
  - Other log suppliers, e.g. a Python application, can use the stand-alone ACS Log Server which provides the generic functionality.

The Centralized Logger service implements the CORBA Telecom Log Service, this is the facility that receives log entries from the entire system and dispatches them to interested clients. Particularly, it implements the Log interface of the Telecom Log Service. Currently ACS is using Telecom Log Service implementation provided by The ACE ORB (TAO), but in a very custom way, using just a minimal subset of the functionality provided by the standard and its implementation.

The Centralized Log uses CORBA for getting the routed logs from the logs suppliers (publishers), applications that use the formatting, buffering and pushing capabilities of ACE API, Java Logging API, the ACS Log Service, etc. as they send log records to it. Thus, the Centralized Logging Service receives a structure in a call to `write_records` from a supplier. The XML string is formatted according to the listing 1. Since the XML string already contains the timestamp information, it should take precedence over the actual current timestamp of the reception of the log entry.

Listing 1. A generic log entry representation in XML

```
<LogEntryType TimeStamp="yyyy-MM-ddThh:mm:ss.fff"
    File="filename" Line="lineno"
    Routine="routine"
    Host="hostname" Process="procname" Thread="threadname"
    Context="context" StackId="stackid" StackLevel="stacklevel"
    LogId="id" Uri="uri"
    Priority="p">
    <Data Name="name">value</Data> log entry message
</LogEntryType>
```

The parsed log entry is then forwarded to the appropriate event channel in the CORBA Notification Service instance, which further distributes the log entries. The choice of the event channel depends on the log entry's type and content. For example, log entries related to debugging could be forwarded to a different event channel than those related to archiving. The logs consumers (clients) can access log entries by subscribing to the event channel of interest as shown in figure 1.

Any application in the system, which may be a container or client, also formats the logs in a much reduced format, which are printed to the standard ouput. During the deployment, every standard output of each deployment unit is redirect to a file. This means that those logs and errors are stored in these files, in colloquial terms, these log files are called "local container log files".

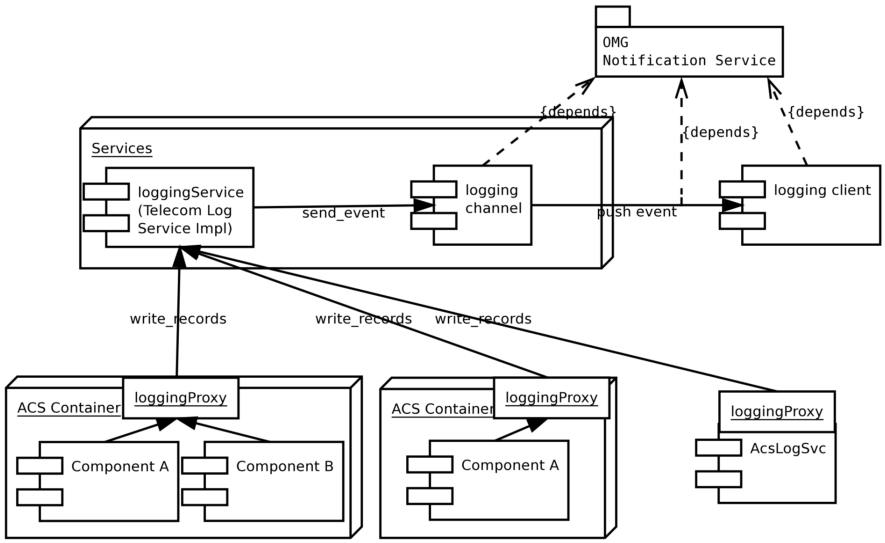


Figure 1. ACS logging service deployment

2.2 Operating System logging

The data acquisition system has more logs which are not part of ACS logging system, those logs corresponds to the hardware infrestructure. ALMA uses the following server deployemnt: 1 server for general network services; 8 server machines where the high level software runs and around 120 diskless machines for hardware control (antennas, correlator computers, timing, etc). All the server machines deployed in the system use Red Hat linux as the Operating system, hence they rely on linux **syslog** to distribute the logs. ALMA has custom configurations to persist diskless machine's logs using syslog.

2.3 Web services logging

Most of the software belonging to OFFLINE software (software which are not related to astronomical data adquisition process), are web-based and compose a critical part of the software infrastructure used in ALMA. The variety of applications in OFFLINE infrastructure is wide: There is a big number of java-based web applications but also, there is a microservice network in which several API's provide a specific functionality for different purposes (i.e: observation state changer service as a REST API). Currently, all OFFLINE infrastructure is being hosted using Docker containers deployed over a swarm cluster.

Each application wrapped on a Docker container produce its logs in their own way and store its respective log files in a specific folder inside a shared volume (NAS storage). In this way, logs persists regardless the container lifecycle and so, logs can be reached even if the container does no longer exist.

3. STORING, RETRIEVING AND QUERYING FOR LOGS

Log analysis plays a critical role not only for problem's troubleshooting but also, on knowing the behaviour of ALMA software at specific time. During observatory operations, problem reports (PR's) belonging to ALMA software are being reported over time on JIRA platform and assigned to SOFTOPS team for the corresponding investigation.[1] As it was mentioned before, ALMA Software includes a wide variety of software, each of those producing logs in different formats, at different rates (logs per second), and storing those logs in different locations so, the investigation through log files was an activity that some times was very time-demanding specially when complex crossing of data were necessary to be made.

3.1 The old times

The situation in terms of the software and its deployment was too rough in terms of thinking about of an automated way to perform log analysis at that moment. Several weaknesses were identified at that moment which caused a considerable overhead every new investigation:

1. **Decentralized log storage:** Even that the ONLINE side of ALMA Software includes a centralized logging (per environment), the rest of the software used in ALMA have its own way to store the logs making it inefficient in both, management and in terms to cross data across applications. In relation with ONLINE software, a significant amount of logs are being produced (aproximately 1500 logs per second) and stored in chunks of XML files of 129MB in a centralized location per environment.[1] For each investigation, the starting point for log querying was to search over those XML files. In terms of OFFLINE side, the result of how and where logs were stored depended in the type of the application. For java web-based applications for example, logs were stored in an standard location per machine (log folder under `$TOMCAT_HOME`) but for other type of applications, log files can be stored in another path and with another format. Issues like log management and crossing log data between applications are an issue if logs are not being stored in a centralized way.

2. **Inefficient log querying tools:** As log files are text-based, the most affordable way to obtain relevant information for an specific investigation was to use system-based text processing tools (i.e: grep, vim, sed, etc.) which even that those tools can be very powerful, those can take a relevant amount of time to parse a heavyweight log file as those tools are not supported by indexes in order to accelerate the query result.

3. **Lack of tools for visualization:** Whenever trends are needed to be analysed, the only way to perform that type of tasks was to implement a custom script to query the relevant log files and perform a visualization. This task is indeed very time consuming during the investigation of an issue in operations.

So, in summary, the situation at that point was unfeasible in terms of think about an automated way to perform log analysis.

3.2 The rise of ELK in ALMA

The necessity of decreasing the overhead time whenever a new investigation is performed and looking forward to automate tasks like issue classification or anomaly detection gave a way to look up how the issues described in Section 3.1 and so, ELK stack emerged as a potential option to be implemented in ALMA.

The first ELK attempt implemented in ALMA was formalized by a project named *LogAAA*[2] which consisted in the deployment and implementation of third-party tools in order to push all ONLINE-related logs into an Elastic database and use Kibana as a tool to perform efficient log queries.

In order to decrease the risk of implementing code inside the exiting ACS logging service in order to push logs directly into *Elastic* database, an in-house implementation (*alogging2mq*) was done in order to create a bridge between the logs produced by the ONLINE infrastructure and ELK stack. This tool acts as a daemon in order to collect (in background) the XML log files produced by ACS and push it into *RabbitMQ* in JSON format (as the main format used in ELK is JSON-based structure). After logs are pushed into *RabbitMQ*, *Logstash* consumes the JSON-based log entries from *RabbitMQ*, performs a parsing of the relevant fields of each log entry and later, ingests it into an *Elastic* database acting as centralized log storage. Finally, *Kibana* was used as a client-based solution to perform queries into *Elastic* database engine, obtain query results and perform visualizations. The previous explanation is detailed as a data-flow diagram in Figure 2.

As it was mentioned before, *LogAAA* was the first attempt in trying to implement a robust logging infrastructure to support ALMA Software and, as a proof-of-concept, the results revealed that the implementation of ELK for log management solved most of the weaknesses that prevented to think about in automated log analysis. The main issue faced during the *LogAAA* implementation was that the amount of logs that could be handled over the existing platform (due storage and computation boundings) was at most 3 days of past history, a value which is too low to jump into the next step (automated log analysis and trending detection).

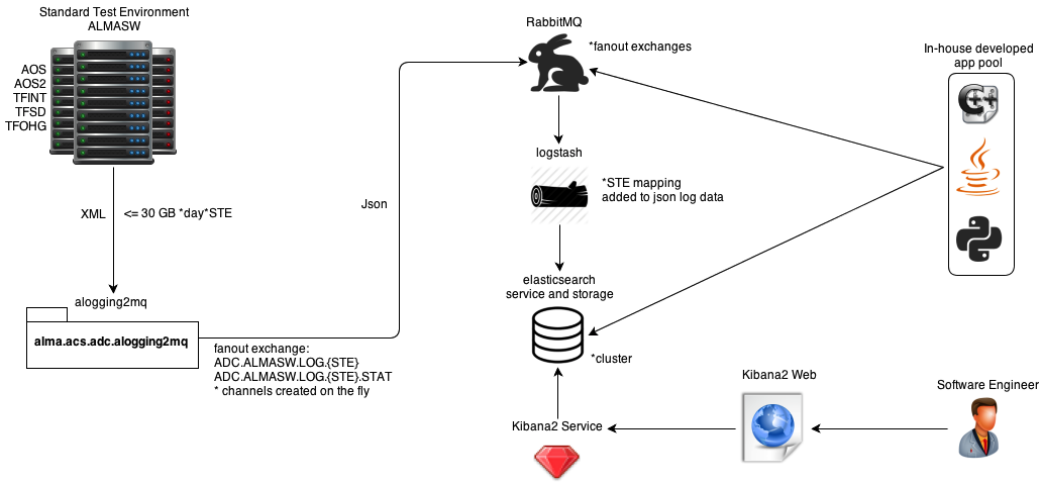


Figure 2. LogAAA deployment diagram.

3.3 ELK as a centralized logging infrastructure in ALMA Software

Even though *LogAAA* did not scale for the expected number of log entries to be handled, the results of that project were promising. Given that situation, extra efforts were made in order to implement a robust cluster-based architecture in order to cope the relevant amount of log entries produced specially in the ONLINE side of ALMA Software. So, the first action performed, was to obtain more capable hardware specifications in order to be able to hold as much logs entries as possible. In addition, a sharded and rebundant database was done through the implementation of an Elastic cluster in order to guarantee high availability and performant indexing.[1] A detailed view of the architecture used to scale the platform can be seen in Figure 3

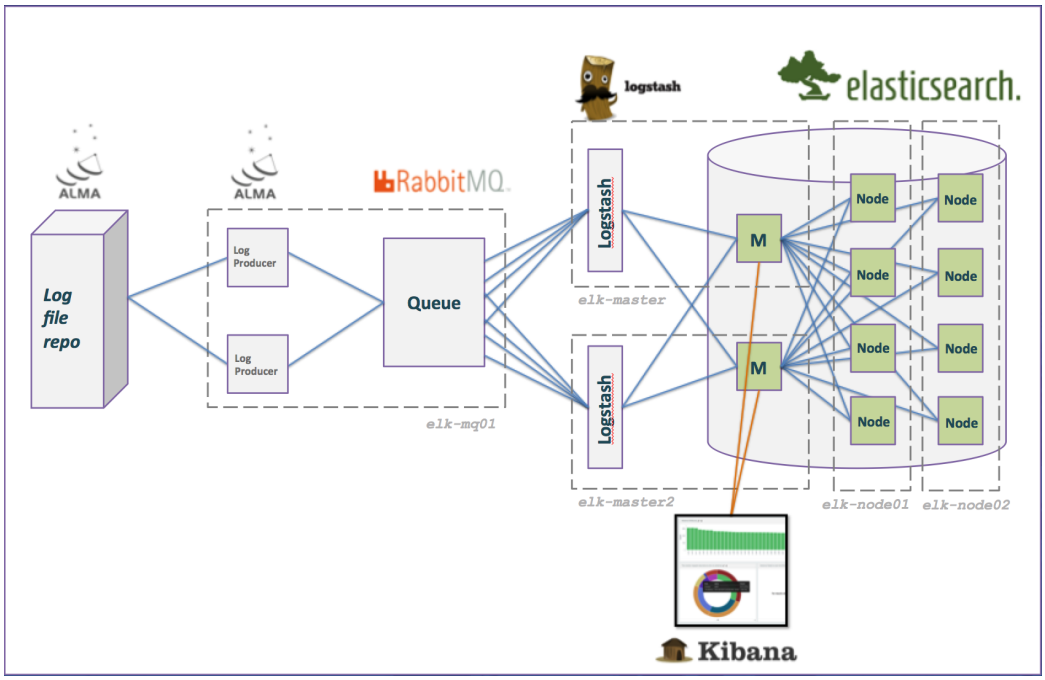


Figure 3. ELK architecture diagram used in ALMA

Those two changes allowed to scale from the storage of just a few days to being able to store one year of logs of ONLINE infrastructure and also being able to perform demanding queries with a considerable time range in

just a couple of seconds.[1]

After being able to reach an important milestone: centralization of a huge amount of log entries of the ONLINE infrastructure of ALMA Software through ELK, also extra efforts were made in order to also include logs produced by the OFFLINE infrastructure. The inclusion of *FileBeat*, open-source software implemented by *Elastic* used to harvest log files into logstash, made the task of including all the data of OFFLINE-related applications without too much extra effort and so, being able to cover the log centralization of almost all operational software applications being used in ALMA. A detailed view of the current dataflow of logs from their source to the final destination can be seen in Figure 4.

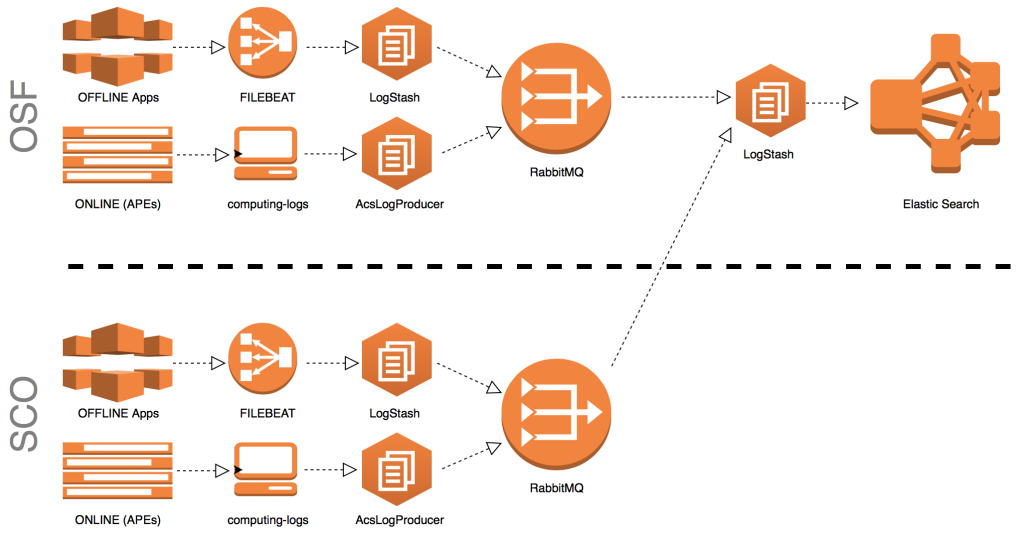


Figure 4. ELK dataflow diagram for logs produced by ALMA Software

All previous improvements in terms of ELK infrastructure gave immediate benefits in the investigation process during operations. The centralization of almost all application logs and being able to perform robust queries in a single web application (*Kibana*) gave a huge speedup to the detection of relevant information needed for each problem report. The implementation of dashboards with the tracking of occurrences of issues that can be identified through complex queries revealed the feasibility to go a step further looking towards to automatically link log occurrences with related problem reports. An example dashboard used in operations is detailed in Figure 5.



Figure 5. Kibana Dashboard used to track known Problem Reports

4. EXTRACTING INFORMATION AND TRENDS

Software logs are used in a variety of scenarios by different groups inside the observatory and externally by developers and testers located in other locations. We make here the distinction between logs generated by software written for ALMA and those generated by operating system (syslog) and existing software (httpd). The discussion below applies to logs generated by ALMA software, though the same kind of analysis is excpected for the later type in the middle term.

The main usage of software logs is during the investigation of problems that appear during operation. These problems are reported in JIRA where the reporter writes as minimum the *timestamp*, *location* and the *observed symptoms*. For a detailed description of this process see.[1] In the early years of ALMA, software logs were used mostly to solve problems for ONLINE software because data acquisition was still stabilizing and the main source of downtime, but during last years OFFLINE applications has growth enough to match the amount of support work needed, specially those apps used for reduction stage and quality checks previous to it. Another scenario where software logs are useful is during software testing. The logs generated during tests done on simulated or production systems are stored in the systems described above for later inspection by testers and developers, and many problems have been detected in this stage before reach production, for example abnormal usage of hardware resources, memory leaks, etc. New features are tested and shown to work as expected using the software logs as well.

There are several types of analysis done over software logs, and they can be grouped by its complexity and the information extracted from the logs. Table 1 summarizes how software logs are analyzed at ALMA. It is worth to note that those analysis are usually combined with OS monitoring systems to provide context regarding CPU usage, memory consumption, swap space and other factors that could have impact on the software operationg the antennas.

Analysis Type	Number of Events	Example
Exploratory	Multiple	Describe a problem by a set of representative logs
Event existence	Single	Java Null Pointer Exception
Value extraction	Single	Measured Voltage
Process analysis	Multiple	Error under specific preconditions
Feature extraction	Multiple	Unexpected delays between events

Table 1. Log analysis types, ordered by complexity

A warning for those who will incorporate log analysis is given here: in the case of ALMA, as in many young observatory facilities, the software evolves quickly and both the structure and the information contained in logs may vary over the time, forcing to update the log analysis tools. There is a need of a good communication between analysts and developers to make the logging messages as stable as possible.

4.1 Exploratory

The first step of log analysis is to manually explore the logs. The common use case is to start in the timestamp when a problem happens, investigate the context and find a proper set of logs that describes the behavior in a concise way. In ALMA context, a typical problem is reported describing its symptoms. Sometimes it contains a sample of logs including the errors but it is not uncommon to see a vague depiction of the failure with no clues of the underlying error. Another common example is to have just the last symptom, and a deep investigation is needed to understand where the problem was originated. Note that this is not a fault of the reporter, but a consequence of the complexity of the system.

To illustrate this, a given problem reporting "*Observation fails*" could be traced to, for example, "*antenna X did not respond on time*" but this is found only by inspecting the context around the reported time of occurrence. After careful analysis of a problem the symptoms are narrowed to its root cause and a detailed set of logs is written in JIRA as evidence. We have access to the source code of ALMA software, and we normally include snippets of code in the detalied description of the root cause analysis to make explicit where the problem is being originating. Once all the information is collected and organized, the developer takes control of the issue.

4.2 Event Existence

There are behaviors described by a simple event log. Typical examples are errors generated by the compilers like Java Null Pointer Exception, OutOfMemory Error, and errors written manually by developers when an expected error condition is found, as in "Time out moving the antenna". This type of errors can have parameters inside the text associated with them: "ACD took too long to move: *5.5* seconds" but in general they are described by the invariant part on it: "ACD took too long to move". Not only errors are detected in this way, for example a developer can provide a patch that includes event logs written for testing purposes, for example to prove that a specific path inside an if/else is actually reached.

The most direct analysis is search for a known event. Text tools as *grep* are enough for this task if the size of the log file is reasonable. However, if long range analysis is needed then more sophisticated tools as Elasticsearch are needed. A common scenario is to find a problem described by a single event, and ask the question "How many times this has been happening in the last 3 months", that would imply do a grep over hundreds of Gigabytes in contrast to a simple Kibana search that throws results in the order of seconds.

Dashboards can be set up with Kibana to monitor desired single events and provide early alert of known problems. Also, the aggregation capabilities of Elasticsearch allows to take the events and display the count of them in graphs. Extensive examples are provided in[1] and in Figure 5.

4.3 Value Extraction

Some developers writes the results of calculations or physical measures inside the body of the event log. This data could be categorical, like "on/off", or numerical, like measured current. Extracting those values depends on the way that the developer choose to write this data, but usually specialized mechanisms are needed to be able to work with this kind of information. As it is computationally inefficient to have a parser listening the text file storing the log, the recommended way is to have a query on the log database that parses and store the values in a suitable format. At ALMA we use the Elasticsearch API to return desired logs either in Python or Bash, and then parse this result to create graphs or store in timeseries databases that allows long term analysis of these values. This kind of work have been useful for hardware engineers monitoring specific parameters, and developers who need to inspect the result of intermediatte steps of their calculations, available at debug level.

4.4 Process Analysis

The information of process execution is only stored in software logs, as opposite with previous analysis when a similar information could be extracted by other meanse. While pure software operations can be analyzed from a process point of view, the real impact of this analysis is studying hardware behavior controlled by the software. As most of the hardware used in astronomical facilites are unique, like at ALMA, to gain deep understanding on its behavior is of uttermost importance for developers and operation engineers.

The most spread approach used at ALMA is that given that the source code is already known to ALMA engineers, it is possible to investigate whether a set of event logs around a problem follows an expected path. Another kind of models used in troubleshooting is an undesired behavior triggered after a set of preconditions are met. If the logs coincides with a path in the model, then the failure is considered explained. In the following section it will be shown works towards automation of this type of analysis.

4.5 Feature Extraction

This is another advanced analysis whose goal is to generate new information (features) from some relation between events in the whole log. As an example, bad timing or delays between events are implicit in the timestamps but are hard to find. They becomes explicit only when a substraction between two consequtive timestamps are done. Further examples are provided in the following section.

5. ADVANCED TOOLS FOR LOG ANALYSIS

The ALMA software logging infrastructure described in the previous sections has not only made possible to have a better understanding of the distributed system and improve the troubleshooting process inside each one of its subsystems. It also has facilitated the proposal and design of many additional tools and solutions to further automate tasks performed on a daily basis by the ALMA Software Operations group. In this section we will briefly describe some of the data analysis tools ann techniques that have been, or are in the process of being, implemented and tested as useful additions to the ALMA logging infrastructure.

5.1 Standarizing Log Messages, or Giving Colors to Logs

The ALMA Software infrastructure is composed of thousands of hardware devices, controled by software written in several languages, using many frameworks and running on more than one hundred machines (virtual and physical). One of the consequences of this heterogeneous software and hardware ecosystem is that the log messages generated by the software have many different formats, extensions and semantic components. Therefore, one of the first problems that we face when trying to perform log analysis is how can we compare or relate different log messages.

Generally speaking, most of the log messages are in some kind of plain text format, or at least can be straightforwardly converted to it. Then, it follows that most of them can be analyzed using standard text mining or processing techniques. Some of them have as their key step transforming the collection of text documents they receive as inputs into mathematical representations (such as vectors) which are easily compared using different kinds of metrics. However, for most of the practical use cases seen in the context of the ALMA Software log analysis, it is enough to be able to compare two pieces of text in a binary manner; e.g. being able to say two different log messages correspond to a same hardware-triggered event or not.

Consider, for instance, the following case. Here is a collection of log messages received by the ACS Master Controller:

```
INFO [ACACORR_MASTER_COMP] starting component ACACORR/CDPCIF/NODE_22
INFO [ACACORR_MASTER_COMP] submitted reference task for component ACACORR/CDPCIF/NODE_19 (ref=null)
INFO [ACACORR_MASTER_COMP] starting component ACACORR/CDPCIF/NODE_20
INFO [ACACORR_MASTER_COMP] submitted reference task for component ACACORR/CDPCIF/NODE_23 (ref=null)
INFO [ACACORR_MASTER_COMP] starting component ACACORR/CDPCIF/NODE_19
INFO [ACACORR_MASTER_COMP] submitted reference task for component ACACORR/CDPCIF/NODE_17 (ref=null)
INFO [ACACORR_MASTER_COMP] starting component ACACORR/CDPCIF/NODE_23
INFO [ACACORR_MASTER_COMP] submitted reference task for component ACACORR/CDPCIF/NODE_21 (ref=null)
```

In this case there are clearly two types of events being logged in this set of 8 messages. However, if a simple algorithm compares this messages in a straightforward way (for instance, word by word), the result will be that this set is made out of 8 different messages; even though for any human reader these messages are clearly quite similar between one another.

In order to sort this problem, we could define the following two template texts with placeholders:

```
INFO [ACACORR_MASTER_COMP] starting component ACACORR/CDPCIF/NODE_#
INFO [ACACORR_MASTER_COMP] submitted reference task for component ACACORR/CDPCIF/NODE_# (ref=null)
```

where "#" stands for any string of digits. These two basic templates can be given as an input to any algorithm capable of matching regular expressions, and it will come up with the result that 4 messages of the previous set match with the first template and the other 4 with the second. If we identify a template with a particular event, then we now have a simple way to characterize any log composed of a set of messages by the events it describes.

This technique, called “coloring”, has the benefit of being very simple in its application once we have a predefined set of templates, or “colors”, to apply to software logs, and it also allows to represent extensive logs as vectors of very low dimension. The problem of coming up with an efficient set of colors, however, is not an easy one; but even simple heuristics have given very good results in the context of the ALMA Software log analysis.

It is part of a future plan to try more comprehensive solutions to characterize software logs in this context, such as Vector Space Models and Latent Semantic Analysis, and compare their performance and efficiency to the solutions already implemented.

5.2 Delay Detection Based on Software Logs

The interaction of the different subsystems of the ALMA Software, as is in the case with any distributed system, implies very complex intertwining of events distributed in time. Many of the interactions between software components are expected to take place in specific ranges of time, and therefore are regulated by mechanisms such as time-defined exeptions and timeouts. It is, then, not only important to characterize the kinds of events that take place as part of daily operations, but also the time lapses between these. One kind of problem that is common is when unexpected delays appears during the execution of a process.

For instance, if there is a delay in the execution of an observation, but it is not interrupted because of this, then a problem can be triggered in a further execution step, and its root cause go unnoticed until it is specifically searched in the context of a problem investigation. Because of the large amount of events taking place at an arbitrary moment of an observation, the problem of detecting such delays is hard to be resolved manually; and therefore an automated approach is needed. That is why a methodology was developed in order to profile the normal delays between steps of a process inferred by the events it generates, and then use that information for label new events as outliers or abnormal. A more detailed analysis is shown in "A Novelty Method for Delay Detection based on Software Logs", presented at ADASS XXVII.[3]

5.3 Supervised Clasification of Software Problem Reports

Throughout the daily normal operation of the ALMA Observatory, there is a large quantity of software related problems, which are usually reported as Problem Report tickets in the project management software *JIRA*. A team of software operations engineers analyze those PR tickets and, from the information obtained in an investigation process, a kind of ticket called *ICT*, which is addressed to developers, is created in order to implement a solution.

However, in several cases, the initial PR ticket is associated by the investigation team to a preexistent ICT ticket, indicating that the issue described in the PR ticket has already been seen in the past and is being, or has been, addressed by a developer. Therefore, in most cases the investigation process of a PR ticket starts by a search of previous ICT tickets that may be related to the issue at hand. This process is normally aided by the presence of portions of relevant software logs in the contents of the ticket, which can then be related to portions of logs present in other PR or ICT tickets.

For cases like these it became aparent the potential usefulness of an automatic tool to aid in the process of investigating a PR ticket and relating it to a pre-existent ICT ticket, in the form of a scored recommendation to the software operations engineer, so he can hava at first glance the information of where to start looking for clues in the troubleshooting process.

Figure 6 shows a diagram with the feature extraction process. With the help of the coloring procedure described in previous sections, the logs were represented as sets of features, which were then processed by a Association Rule Mining algorithm, in order to extract rules to relate the presence of certain sets of events to a particular ICT ticket with an associated probability.

5.4 Complex Problem Detection by Stateful Models

Process Mining[4] is a relatively new subfield of data science that provides analytical tools to combine process definition with event logs. Two main families of those tools are directly applicable to process analysis at ALMA:

- Conformance checking: address the problem of match an event log against a provided model.
- Process discovery: Models inferred using algorithms from examples of event logs.

The conformance checking problem can be understood as a model that start in an initial state, follows a path by reading event logs, and eventually reach a final state that is interpreted that the overall behavior described in the model was met. Experiments has been done with finite state machines (FSM), started in.[2] The current state of FSM at ALMA includes a set of Python classes to connect with Elasticsearch, do model definition using JSON, do the match with a provided set of log and display an alarm when the final state is reached. The results

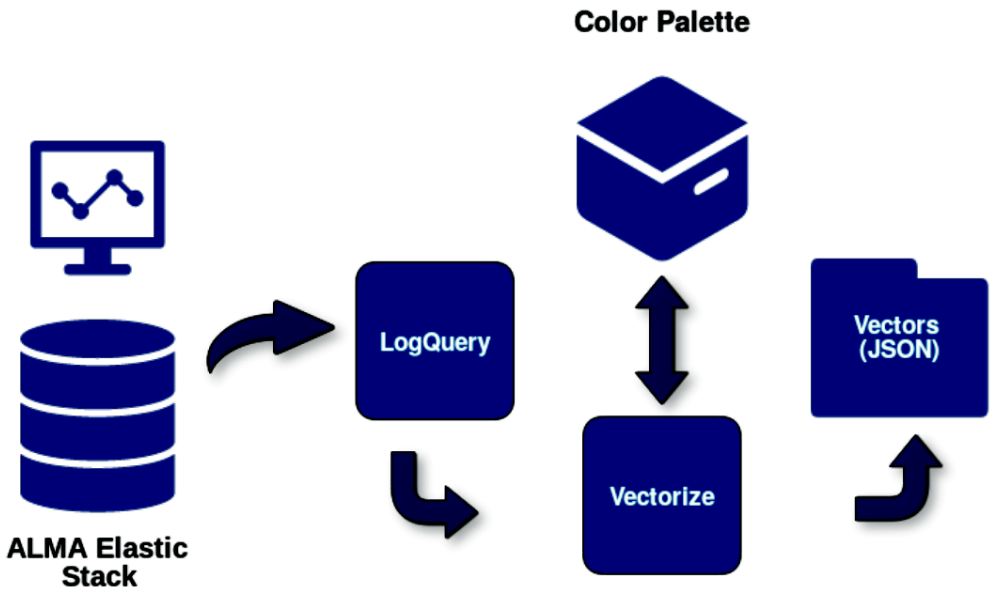


Figure 6. Log feature extraction process.

are promising, several models describing complex problems has been implemented and used to find some hard problems in an automatic way. However, to write one of those model could take some time that usually is not available in day to day operation, then our next step is to implement Process Discovery techniques to have some models inferred from real problems found inside the software logs.

ACKNOWLEDGMENTS

The Atacama Large Millimeter/submillimeter Array (ALMA), an international astronomy facility, is a partnership of the European Organisation for Astronomical Research in the Southern Hemisphere (ESO), the U.S. National Science Foundation (NSF) and the National Institutes of Natural Sciences (NINS) of Japan in cooperation with the Republic of Chile. ALMA is funded by ESO on behalf of its Member States, by NSF in cooperation with the National Research Council of Canada (NRC) and the National Science Council of Taiwan (NSC) and by NINS in cooperation with the Academia Sinica (AS) in Taiwan and the Korea Astronomy and Space Science Institute (KASI). ALMA continued development and operations are led by ESO on behalf of its Member States; by the National Radio Astronomy Observatory (NRAO), managed by Associated Universities, Inc. (AUI), on behalf of North America; and by the National Astronomical Observatory of Japan (NAOJ) on behalf of East Asia.

REFERENCES

[1] Gil, J., Reveco, J., and Shen, T.-C., "Operational logs analysis at alma observatory based on elk stack," (06 2016).

[2] Juan Pablo Gil, Alexis Tejeda, T.-C. S. N. S., "Unveiling alma software behavior using a decoupled log analysis framework," (2014).

[3] Gil, J., Russo, B., and Avila, A., "A novelty method for delay detection based on software logs," in [*ADASS XXVII*], TBD, ed., *ASP Conf. Ser.* **TBD**, TBD, ASP, San Francisco (2018).

[4] Van der Aalst, W. M., "Process mining: Discovery, conformance and enhancement of business processes," in [*Process Mining: Discovery, Conformance and Enhancement of Business Processes*], Springer Publishing Company, Incorporated (2011).